Formal Modeling and Analysis of Metric-Timed and Multi-Agent Normative Specifications of Actions

Karam Younes Kharraz

Oral examination for the dissertation submitted to obtain
the Dr. rer. nat degree

29 April 2026

First jury:	Prof. Dr. Martin Leucker
Second jury:	Prof. Dr. Gordon J. Pace
Head of the commission:	Prof. Dr. Stefan Fischer

Motivation: The Cost of Manual Compliance

What are Normative Specifications?

- A normative specification is a document or clause that outlines mandatory requirements, technical standards, or protocols that must be followed for compliance.
- Contracts, laws, and regulations are examples of normative specifications.

The Scaling Problem

- Regulatory burden costs the global economy trillions of dollars annually:**3.5 % of Europe's GDP**.
- The manual management of normative specifications is a time-consuming and error-prone process.

The Solution: Legaltech

- Legaltech, or legal technology, refers to the application of software and digital tools to provide legal services and support the justice system.

Motivation: The Cost of Manual Compliance

What are Normative Specifications?

- A normative specification is a document or clause that outlines mandatory requirements, technical standards, or protocols that must be followed for compliance.
- Contracts, laws, and regulations are examples of normative specifications.

The Scaling Problem

- Regulatory burden costs the global economy trillions of dollars annually:**3.5 % of Europe's GDP**.
- The manual management of normative specifications is a time-consuming and error-prone process.

The Solution: Legaltech

- Legaltech, or legal technology, refers to the application of software and digital tools to provide legal services and support the justice system.

Motivation: The Cost of Manual Compliance

What are Normative Specifications?

- A normative specification is a document or clause that outlines mandatory requirements, technical standards, or protocols that must be followed for compliance.
- Contracts, laws, and regulations are examples of normative specifications.

The Scaling Problem

- Regulatory burden costs the global economy trillions of dollars annually:**3.5 % of Europe's GDP**.
- The manual management of normative specifications is a time-consuming and error-prone process.

The Solution: Legaltech

- Legaltech, or legal technology, refers to the application of software and digital tools to provide legal services and support the justice system.

Legaltech between probabilistic AI and formal methods

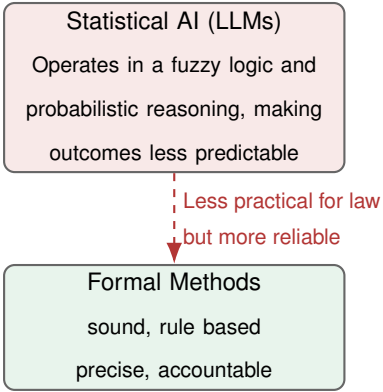
LLM in legaltech

According to a new survey of over 200 inhouse and law firm lawyers [a], 69.7% of legal outputs still require targeted edits or extensive rework.

[a] https://www.artificiallawyer.com/2026/04/07/

The Formal Methods Alternative

Legislation as code aims to treat specifications as mathematical objects, where questions about behavior yield provable verdicts rather than statistical guesses.



Legaltech between probabilistic AI and formal methods

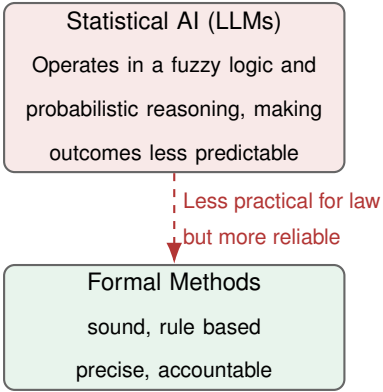
LLM in legaltech

According to a new survey of over 200 inhouse and law firm lawyers [a], 69.7% of legal outputs still require targeted edits or extensive rework.

[a] https://www.artificiallawyer.com/2026/04/07/

The Formal Methods Alternative

Legislation as code aims to treat specifications as mathematical objects, where questions about behavior yield provable verdicts rather than statistical guesses.



Thesis Vision: The two families of formal methods applied to Normative reasoning

Thesis Contribution: two formal pipelines for timed normative conflicts and blame-tracking runtime monitors in multi-agent settings.

Before Adoption — Static Analysis

Reasoning about conflicts in metric timed normative specifications.

Problem: Conflicting norms *hide* inside complex temporal conditions, we suggest a novel characterization of timed normative conflicts.

During Enforcement — Runtime Monitoring

Perpetual Blame Monitoring in multi-agent contracts.

Problem: Contracts specify collaborative multi-agent behavior that may run indefinitely, requiring:

- Monitoring that also tracks *obstructions*, not only non-performance, and
- Per-agent *cumulative blame attribution* over the whole contract execution.

Part I

Reasoning about Metric-Timed Normative Conflicts

Micro Metric-Timed Normative Logic μMTNL

Intuitive understanding of normative conflicts

The Fundamental Intuition

"Normative conflicts occur when complying with one norm can lead to noncompliance with another."

— H. Kelsen, *General Theory of Norms* (1991)

Deontic Conflicts in Dynamic Settings over time

"A deontic conflict occurs when a set of obligations is jointly unsatisfiable over time, meaning that no trace can be constructed that complies with every obligation in the set."

— S. Colombo Tosatto et al., *DEON* (2014)

Conflicts via Satisfiability

"A normative conflict occurs when triggering a normative rule, either always or in some feasible situation, inevitably leads to the violation of one or more rules, because the rule set imposes incompatible requirements."

— N. Feng et al., *ICSE* (2024)

Intuitive understanding of normative conflicts

The Fundamental Intuition

"Normative conflicts occur when complying with one norm can lead to noncompliance with another."

— H. Kelsen, *General Theory of Norms* (1991)

Deontic Conflicts in Dynamic Settings over time

"A deontic conflict occurs when a set of obligations is jointly unsatisfiable over time, meaning that no trace can be constructed that complies with every obligation in the set."

— S. Colombo Tosatto et al., *DEON* (2014)

Conflicts via Satisfiability

"A normative conflict occurs when triggering a normative rule, either always or in some feasible situation, inevitably leads to the violation of one or more rules, because the rule set imposes incompatible requirements."

— N. Feng et al., *ICSE* (2024)

Intuitive understanding of normative conflicts

The Fundamental Intuition

"Normative conflicts occur when complying with one norm can lead to noncompliance with another."

— H. Kelsen, *General Theory of Norms* (1991)

Deontic Conflicts in Dynamic Settings over time

"A deontic conflict occurs when a set of obligations is jointly unsatisfiable over time, meaning that no trace can be constructed that complies with every obligation in the set."

— S. Colombo Tosatto et al., *DEON* (2014)

Conflicts via Satisfiability

"A normative conflict occurs when triggering a normative rule, either always or in some feasible situation, inevitably leads to the violation of one or more rules, because the rule set imposes incompatible requirements."

— N. Feng et al., *ICSE* (2024)

Motivation: Inpterpreting Timed norms as a Planning Problem

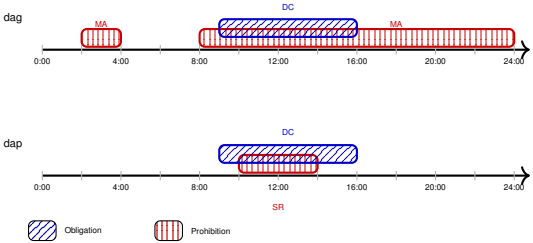
Use case: Goods delivery to a harbour.

- DC: Deliver between 09:00–16:00 to gate **or** parking.
- SR: Parking **prohibited** 10:00–14:00.
- MA: Gate **prohibited** 02:00–04:00, 08:00–24:00.

Conflicts are not logical contradictions in isolation: they arise from *overlapping activation windows* and *single-action capacity*.

Conflict at a specific time point t:

1. **Deontic:** Two contradictory obligations/prohibitions specified at the same t.
2. **Ontic:** Two obliged actions cannot be performed at the same time t.



SAT result: DC ⊓ SR ⊓ MA is satisfiable — model: dap at $t = 9$.

Syntactic definition and analysis of conflicts in μMTNL

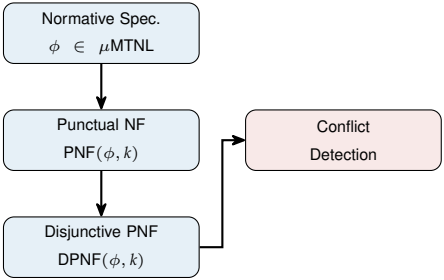
Punctual Conflicts

Conflicts are identified syntactically at individual time points t:

1. **P-Deon-Conf:** $O^{\{[t,t]\}}\, a \sqcap F^{\{[t,t]\}}\, a$
2. **P-Ont-Conf:** $O^{\{[t,t]\}}\, a \sqcap O^{\{[t,t]\}}\, b$

As literals in formulas are not always punctual, we introduce the DPNF that decomposes time intervals into punctual atoms, allowing us to detect conflicts at the punctual level.

The Syntactic Analysis Pipeline



- Structural invariants literal ancestor preservation.
- Exponential worst-case blow-up in the size of DPNF(ϕ, x).

Example of conflict analysis

Step	Value	Size
ϕ	$O^{\{[1,2]\}}\,\mathsf{dag} \sqcap F^{\{[2,5]\}}\,\mathsf{dag} \sqcap O^{\{[1,2]\}}\,\mathsf{pick}$	14
$\mathsf{PNF}(\phi, 2)$	$\left(O^{\{[1,1]\}}\,\mathsf{dag} \sqcup O^{\{[2,2]\}}\,\mathsf{dag}\right) \sqcap \left(F^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[3,5]\}}\,\mathsf{dag}\right) \sqcap \left(O^{\{[1,1]\}}\,\mathsf{pick} \sqcup O^{\{[2,2]\}}\,\mathsf{pick}\right)$	33
$D\mathsf{PNF}(\phi, 2))$	$\left(O^{\{[1,1]\}}\,\mathsf{dag} \sqcap F^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[3,5]\}}\,\mathsf{dag} \sqcap O^{\{[1,1]\}}\,\mathsf{pick}\right)\ \underline{pt_1}$ $\sqcup\ \left(O^{\{[1,1]\}}\,\mathsf{dag} \sqcap F^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[3,5]\}}\,\mathsf{dag} \sqcap O^{\{[2,2]\}}\,\mathsf{pick}\right)\ \underline{pt_2}$ $\sqcup\ \left(O^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[3,5]\}}\,\mathsf{dag} \sqcap O^{\{[1,1]\}}\,\mathsf{pick}\right)\ \underline{pt_3}$ $\sqcup\ \left(O^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[3,5]\}}\,\mathsf{dag} \sqcap O^{\{[2,2]\}}\,\mathsf{pick}\right)\ \underline{pt_4}$	76
$\mathcal{PC}(\phi)$	$(\mathsf{pt}_1, \{O^{\{[1,1]\}}\,\mathsf{dag} \sqcap O^{\{[1,1]\}}\,\mathsf{pick}\}),$ $(\mathsf{pt}_3, \{O^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[2,2]\}}\,\mathsf{dag}\})$ $(\mathsf{pt}_4, \{O^{\{[2,2]\}}\,\mathsf{dag} \sqcap F^{\{[2,2]\}}\,\mathsf{dag}, O^{\{[2,2]\}}\,\mathsf{dag} \sqcap O^{\{[2,2]\}}\,\mathsf{pick}\})$	–

Key results through conflict density

Conflict Density

$$\mathsf{C}_\delta(\phi) := \frac{|\text{CPT in DPNF}(\phi)|}{|\text{PT in DPNF}(\phi)|}$$

- $\mathsf{C}_\delta = 0$: conflict-free.
- $\mathsf{C}_\delta = 1$: fully conflicting.
- Use case: $\mathsf{C}_\delta(\text{UC}) = 75\%$.

Conflicts vs unsatisfiability

Key results:

- ϕ is unsat $\iff$ $\mathsf{C}_\delta(\phi) = 1$
- Punctual conflicts $\equiv$ Minimal unsatisfiable set (MUS) in the DPNF.

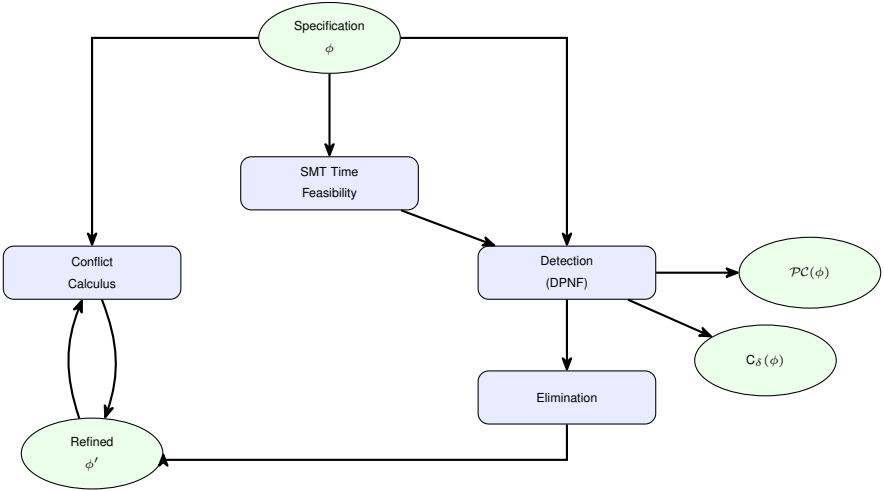
Conflict-Free Rewriting

If $\mathsf{C}_\delta(\phi) < 1$:

$$\exists\, \phi' \equiv \phi, \quad \mathsf{C}_\delta(\phi') = 0$$

- Prune conflicting terms.
- May return a very big size repaired formula.
- Conflict calculus for choosing to partially repair the formula.

Sound and complete Conflict Analysis & Resolution Framework



Part II

Reasoning about Blame in Multi-Party Collaborative Contracts

Two-Agent Collaborative Normative Logic TACNL

Challenges in Monitoring Multi-Party open-ended Contracts

Challenges

- Contract assign a duty to each party, but the outcome depends on their joint behavior.
- A violation may activate a reparation duty rather than a simple failure.
- Contracts are open-ended: they may be terminated or repeated indefinitely.

Running use case: rental contract

- Tenant must pay rent every month.
- Missed rent payement activates a late-fee reparation.
- Landlord must guarantee occupancy.
- Payement and occupency are repeated in a montly basis until a termination notification is submitted.

Part II Goal: Design a monitoring framework for multi-party collaboration.

- Distinguish agent attempts from tasl outcome for precise blame attribution.
- Support monitoring over the whole lifecycle of the contract.

The two agent collaborative normative logic TACNL

Primitives

$\mathbf{O}_p(a)$: obligation

$\mathbf{F}_p(a)$: prohibition

$\mathbf{P}_p(a)$: power

$a \in \Sigma_C$ is a collab action and $p \in \{1, 2\}$ is party.

Compositional control flow

$C \wedge C$: parallel duties

$C \blacktriangleright C$: contrary-to-duty repair

$\langle\!\langle re \rangle\!\rangle C$: event-triggered activation

$\lceil re \rceil C$: termination / bounded applicability

$\mathbf{Rep}(C)$: open-ended repetition

Rental contract encoding

$$C_1 := \mathbf{O}_1(\text{PAY_R})$$

$$C_2 := \mathbf{P}_1(\text{OCC})$$

$$C_3 := \mathbf{O}_1(\text{PAY_R}) \blacktriangleright \mathbf{O}_1(\text{PAY_F})$$

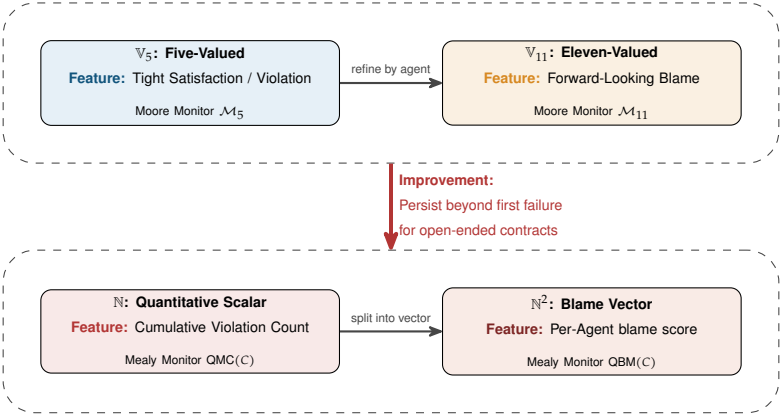
$$C_4 := \langle\!\langle \{\mathbf{Notif_R}^{(1)}\} \rangle\!\rangle \mathbf{O}_2(\text{REPAIR})$$

$$C_{5\to 2} := \lceil \Gamma^+ \cdot \{\mathbf{Notif_T}^{(1)}\} \cdot \Gamma^3 \rceil \mathbf{Rep}(C_2)$$

Together these formulas cover the chapter's core phenomena: ordinary duties, blocked cooperation, repair after violation, event-driven activation, and finite termination of an otherwise open-ended contract.

Progressive semantics for blame attribution

1. Forward-Looking Family (First Decisive Prefix)



2. Quantitative Family (Continuous Evaluation)

Tight Forward Denotational Semantics

Two verdict domains

- $\mathbb{V}_5 = \{?, \top^t, \bot^t, \top^p, \bot^p\}$
- $\mathbb{V}_{11} = \{?, \top^t, \top^p\} \cup \{\bot^t_S, \bot^p_S \mid S \subseteq \{1, 2\}\}$

Why it matters: reparations and triggers switch exactly at the decisive prefix.

Obligation example: $C = \mathbf{O}_1(\text{PAY_R})$

$$\text{PAY_R}^{(1)} \notin A \quad \Rightarrow \quad \mathbb{V}_5 : \bot^t$$

$$\text{PAY_R}^{(1)} \in A,\ \text{PAY_R}^{(2)} \notin A \quad \Rightarrow \quad \mathbb{V}_5 : \bot^t$$

In $\mathbb{V}_5$, both failures are just violations.

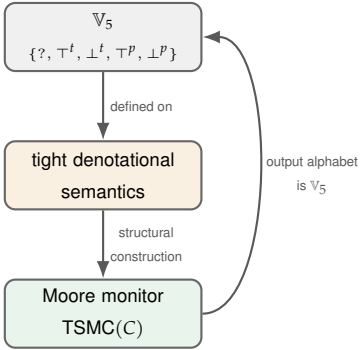
$\mathbb{V}_{11}$ distinguishes blame

$$\text{PAY_R}^{(1)} \notin A \quad \Rightarrow \quad \bot^t_{\{1\}}$$

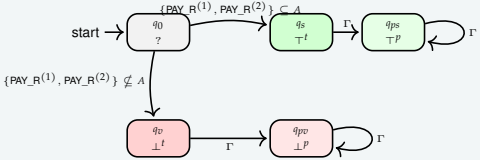
$$\text{PAY_R}^{(1)} \in A,\ \text{PAY_R}^{(2)} \notin A \quad \Rightarrow \quad \bot^t_{\{2\}}$$

Same frontier, but now we know who caused the violation.

Correct-by-Construction Moore Monitors



Example: Moore machine for $\mathbf{O}_1$(PAY_R)



Construction

- Construction à la Thomson.
- Conformance proof to the semantics.

Quantitative violation $[\![]\!]_{\text{qviol}} : \Gamma^* \times \text{TACNL} \to \mathbb{N}$

Two ingredients

$\text{LH}(C)$ tells us *which literal is currently in force.* $\text{Prog}(\langle A \rangle, C)$ tells us *how the contract evolves after the current event.*

Fixpoint intuition

$$[\![\langle A \rangle \cdot \pi, C]\!]_{\text{qviol}} = \underbrace{[\![\langle A \rangle, C]\!]_{\text{qviol}}}_{\text{score now}} + \underbrace{[\![\pi, \text{Prog}(\langle A \rangle, C)]\!]_{\text{qviol}}}_{\text{continue on the residual}}$$

Orange: score the current event. Blue: continue with the residual contract.

Example

Let $C = \mathbf{O}_1(\text{PAY_R}) \blacktriangleright \mathbf{O}_1(\text{PAY_F})$.

$$\pi = \langle A_1 \rangle \cdot \langle A_2 \rangle, \quad A_1 = \{\text{OCC}^{(1)}\}, \quad A_2 = \{\text{PAY_F}^{(12)}\}$$

$$\langle A_1 \rangle : \text{LH}(C) = \mathbf{O}_1(\text{PAY_R}), \quad [\![\langle A_1 \rangle, C]\!]_{\text{qviol}} = 1, \quad \text{Prog}(\langle A_1 \rangle, C) = \mathbf{O}_1(\text{PAY_F})$$

$$\langle A_2 \rangle : \mathbf{LH}(\mathrm{O}_1(\mathbf{PAY_F})) = \mathbf{O}_1(\text{PAY_F}), \quad [\![\langle A_2 \rangle, \mathbf{O}_1(\text{PAY_F})]\!]_{\text{qviol}} = 0$$

$$[\![\pi, C]\!]_{\text{qviol}} = 1 + 0 = 1.$$

Intuition for the Quantitative Violation Monitor QMC(C)

Construction idea

Turn the denotational recursion into a Mealy machine:

$$\mathsf{QMC}(C) = (Q, q_0, \Gamma, \delta, \lambda)$$

where

- $Q = \mathsf{RRC}(C)$ is the set of reachable residual contracts,
- $q_0 = C$ is the initial residual,
- $\delta(q, A) = \mathsf{Prog}(\langle A \rangle, q)$ updates the contract state,
- $\lambda(q, A) = [\![\langle A \rangle, q]\!]_{\mathsf{qviol}}$ emits the local cost.

Finite state Mealy machine

If $\mathsf{RRC}(C)$ is finite, the monitor is finite too.

Execution view

$$q_0 = C \xrightarrow{\langle A_0 \rangle / \lambda(q_0, A_0)} q_1 \xrightarrow{\langle A_1 \rangle / \lambda(q_1, A_1)} q_2 \xrightarrow{\cdots} q_n$$

Each transition does two things:

- advance the residual contract by progression;
- output the score contributed by the current event.

Correctness takeaway

Along the unique run on π,

$$\mathsf{Score}(\mathsf{QMC}(C), \pi) = \sum_{i=0}^{n-1} \lambda(q_i, A_i) = [\![\pi, C]\!]_{\mathsf{qviol}}.$$

So the monitor is an executable realization of the quantitative semantics.

Summary of Contributions

Conflict Analysis (μMTNL)

1. Formal machinery: interval-set logic, DPNF, conflict calculus.
2. Exact characterisation: unsat $\Leftrightarrow$ $\mathsf{C}_\delta(\phi) = 1$.
3. Conflict density metric $\mathsf{C}_\delta(\phi) \in [0, 1]$.
4. Sound & complete conflict-free rewriting $\mathsf{CFR}(\phi)$.
5. Complexity bounds & conciseness analysis.

Contract Monitoring (TACNL)

1. Attempt-based interaction model.
2. TACNL logic with repetition, guards, triggers, reparation.
3. 5-valued tight satisfaction semantics + Moore monitors.
4. 11-valued forward blame semantics + blame monitors.
5. Quantitative scalar ($\mathbb{N}$) and vector ($\mathbb{N}^2$) blame monitors.
6. Full pipeline: denotational $\rightarrow$ operational, with conformance proofs.

Outlook & Future Work

Extending μMTNL:

- Actions with *duration*.
- Out to be (states).
- Conditional & permission literals.
- More ontic conflicts types such as causality and resource conflicts.

Extending TACNL:

- Algorithm and complexity proof for RRC(C).
- n-agent generalisation ($\mathbb{N}^n$ blame vectors).
- Metric time instead of synchronous periods.

MAMTNLAS: Toward a Multi agent Metric Time Normative logic for conflict detection and runtime verification.

Thank you! Questions?

Backup: Lowest Upper Common Operator (LUCO)

Definition

Let $\mathsf{T}(\phi)$ be the Boolean syntax tree of ϕ (internal nodes $\sqcap, \sqcup$; leaves are literals). For literals ℓ_i, ℓ_j each occurring once, $\mathrm{LUCO}_\phi(\ell_i, \ell_j)$ is the *deepest* node v that:

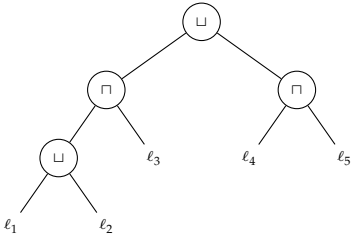
(i) is labeled $\sqcap$ or $\sqcup$;

(ii) lies on both root-to-ℓ_i and root-to-ℓ_j paths;

(iii) no proper descendant of v satisfies (i) and (ii).

Key insight: Two literals can form a punctual conflict *only* when $\mathrm{LUCO} = \sqcap$ (they must appear together in a product term).

Example: $\phi = \big((\ell_1 \sqcup \ell_2) \sqcap \ell_3\big) \sqcup \big(\ell_4 \sqcap \ell_5\big)$



$\mathrm{LUCO}(\ell_1, \ell_2) = \sqcup$ (same branch, disj.)

$\mathrm{LUCO}(\ell_1, \ell_3) = \sqcap$ can conflict

$\mathrm{LUCO}(\ell_4, \ell_5) = \sqcap$ can conflict

$\mathrm{LUCO}(\ell_1, \ell_4) = \sqcup$ (cross-branch)

Theorem: PNF and DNF preserve LUCO— if two ancestors

Backup: Literal Ancestor Preservation (LAP)

Definition

ϕ' satisfies LAP of ϕ if there is a mapping from literals in ϕ' to literals in ϕ such that:

(i) **Downward:** every ℓ' in ϕ' has ancestor ℓ in ϕ with same modality, action, and $\mathfrak{I}(\ell') \subseteq \mathfrak{I}(\ell)$.

(ii) **Upward:** for every ℓ in ϕ, its descendants $\{\ell'_1, \ldots, \ell'_n\}$ in ϕ' recombine under their LUCO to reconstruct ℓ.

Purpose: No new time points are created, none are lost. Normalization only *refines* literals into punctual pieces.

Example:

$$\phi := O^{\{[2,3]\}}\, a \;\sqcap\; F^{\{[2,4]\}}\, a$$

After PNF:

$$\phi' := \left(O^{\{[2,2]\}}\, a \sqcup O^{\{[3,3]\}}\, a\right) \sqcap F^{\{[2,2]\}}\, a \sqcap F^{\{[3,4]\}}\, a$$

Downward ✓:

- $O^{\{[2,2]\}}\, a, O^{\{[3,3]\}}\, a$ descend from $O^{\{[2,3]\}}\, a$
- $F^{\{[2,2]\}}\, a, F^{\{[3,4]\}}\, a$ descend from $F^{\{[2,4]\}}\, a$

Upward ✓:

- $O^{\{[2,2]\}}\, a \sqcup O^{\{[3,3]\}}\, a \equiv O^{\{[2,3]\}}\, a$
- $F^{\{[2,2]\}}\, a \sqcap F^{\{[3,4]\}}\, a \equiv F^{\{[2,4]\}}\, a$

Together: LUCO + LAP guarantee that DPNF normalization ex-

Backup: Conflict-Free Rewriting — Positive & Negative Examples

After conflict extraction: prune all conflicting product terms $\Rightarrow$ CFR(ϕ).

Positive: Compact rewriting exists

$$\mathrm{UC}_2' := O^{\{[9,16]\}}\,\mathsf{dap} \sqcap \underbrace{F^{\{[10,14]\}}\,\mathsf{dap}}_{\text{SR}} \sqcap \underbrace{F^{\{[2,4],[8,24]\}}\,\mathsf{dag}}_{\text{MA}}$$

Only times $t \in \{9, 15, 16\}$ are conflict-free.

Verbose CFR(UC_2') has size 94, but merging via **ObDis**:

$$\mathrm{UC}_2'' := O^{\{[9,9],[15,16]\}}\,\mathsf{dap} \sqcap \mathsf{SR} \sqcap \mathsf{MA}$$

Size 18 vs. original 16. ✓

Negative: Exponential blow-up

$$\phi := O^{\{[0,3]\}}\,\mathsf{a} \sqcap O^{\{[0,3]\}}\,\mathsf{b} \sqcap O^{\{[0,3]\}}\,\mathsf{c}$$

Three obligations over 4 time points.

Step	Size
ϕ	18
PNF($\phi, 3$)	59
DNF(PNF($\phi, 3$))	959
CFR(ϕ) (24 PT)	359
Blow-up	$\approx 20\times$

Backup: Conflict Calculus for μMTNL

Eleven inference rules for big-step reasoning — no DPNF expansion required.

Rule	Condition	Conclusion
(O-Union)	$\mathfrak{I} = \mathfrak{I}_1 \cup \mathfrak{I}_2$	$O^{\mathfrak{I}_1}\, a \sqcup O^{\mathfrak{I}_2}\, a \equiv O^{\mathfrak{I}}\, a$
(F-Union)	$\mathfrak{I} = \mathfrak{I}_1 \cup \mathfrak{I}_2$	$F^{\mathfrak{I}_1}\, a \sqcup F^{\mathfrak{I}_2}\, a \equiv F^{\mathfrak{I}}\, a$
(Distr1L)		$\phi_1 \sqcap (\phi_2 \sqcup \phi_3) \equiv (\phi_1 \sqcap \phi_2) \sqcup (\phi_1 \sqcap \phi_3)$
($\equiv \sqcap$)	$\phi_1 \equiv \phi_1'$	$\phi_1 \sqcap \phi_2 \equiv \phi_1' \sqcap \phi_2$
($\bot \equiv$)	$\phi \equiv \phi',\ \phi' \vdash \bot$	$\phi \vdash \bot$
($\bot \sqcap$)	$\phi \vdash \bot$	$\phi \sqcap \phi' \vdash \bot$
($\bot \sqcup$)	$\phi \vdash \bot,\ \phi' \vdash \bot$	$\phi \sqcup \phi' \vdash \bot$
($\sqcup \bot$)	$\phi' \vdash \bot$	$\phi \sqcup \phi' \equiv \phi$
(T-DeoConf)	$\mathfrak{I} \subseteq \mathfrak{I}'$	$O^{\mathfrak{I}}\, a \sqcap F^{\mathfrak{I}'}\, a \vdash \bot$
(T-DeoConf)	[illegible]	[illegible]

Backup: Step 1 — Metric-Timed Traces → Periodic Set Traces

Input: Raw metric-timed traces over 120 days (tenant = agent 1, landlord = agent 2).

$$\tau_1 = \langle(\text{pay_rent}, 5),\ (\text{start_occ}, 35),\ (\text{pay_rent}, 36),\ (\text{pay_rent_f}, 95)\rangle$$

$$\tau_2 = \langle(\text{grant_occ}, 1),\ (\text{ack_pay}, 6),\ (\text{ack_pay}, 37),\ (\text{ref_pay}, 95),\ (\text{change_lock}, 96)\rangle$$

Agg_{30}: partition $[1, 120]$ into $I_1 = [1, 30]$, $I_2 = [31, 60]$, $I_3 = [61, 90]$, $I_4 = [91, 120]$.
Group each event (a, t) into the period I_k containing t, collect action names as sets.

Output: Periodic set traces π_1, π_2:

$$\pi_1 = \langle\underbrace{\{\text{pay_rent}\}}_{I_1},\ \underbrace{\{\text{start_occ},\ \text{pay_rent}\}}_{I_2},\ \underbrace{\emptyset}_{I_3},\ \underbrace{\{\text{pay_rent_f}\}}_{I_4}\rangle$$

$$\pi_2 = \langle\underbrace{\{\text{grant_occ},\ \text{ack_pay}\}}_{I_1},\ \underbrace{\{\text{ack_pay}\}}_{I_2},\ \underbrace{\emptyset}_{I_3},\ \underbrace{\{\text{ref_pay},\ \text{change_lock}\}}_{I_4}\rangle$$

Key property: Within each period, temporal ordering is abstracted away — only *presence* matters. Between periods, the sequential structure is preserved by the index k.

Backup: Step 2 — Collaboration Alphabet & Tagged Traces

Enabler/Blocker mapping (from the contract specification):

$$E_1(\text{PAY_R}) = \{\text{pay_rent}\} \quad E_2(\text{PAY_R}) = \{\text{ack_pay}\}, \quad B_2(\text{PAY_R}) = \{\text{ref_pay}\}$$
$$E_1(\text{PAY_F}) = \{\text{pay_rent_f}\}$$
$$E_1(\text{OCC}) = \{\text{start_occ}\} \quad E_2(\text{OCC}) = \{\text{grant_occ}\}, \quad B_2(\text{OCC}) = \{\text{change_lock}\}$$

Replace concrete actions with collaboration symbols. Enablers → positive contribution; blockers → $\emptyset$ for that symbol.
Collaboration alphabet: $\Sigma_C = \{\text{PAY_R}, \text{PAY_F}, \text{OCC}\}$. Tag each with agent superscript.

Output: Tagged collaboration traces π_1^C, π_2^C:

$$\pi_1^C = \langle \underbrace{\{\text{PAY_R}^{(1)}\}}_{l_1}, \underbrace{\{\text{OCC}^{(1)}, \text{PAY_R}^{(1)}\}}_{l_2}, \underbrace{\{\text{OCC}^{(1)}\}}_{l_3}, \underbrace{\{\text{PAY_F}^{(1)}\}}_{l_4} \rangle$$

$$\pi_2^C = \langle \underbrace{\{\text{OCC}^{(2)}, \text{PAY_R}^{(2)}\}}_{l_1}, \underbrace{\{\text{OCC}^{(2)}, \text{PAY_R}^{(2)}\}}_{l_2}, \underbrace{\{\text{OCC}^{(2)}\}}_{l_3}, \underbrace{\emptyset}_{l_4} \rangle$$

Reading per month: